Router par la preuve et s'abstenir à temps : allocation adaptative du calcul et de la recherche documentaire pour des LLM locaux

Une évaluation pré-enregistrée de la qualité, de la latence et de l'énergie sur matériel local

Alae Lahbichi – Étudiant en 5e année IA/Data

a.lahbichi2004@gmail.com

Code et données : github.com/AlaeLahbichi/Adaptive-Compute-and-RAG-for-Efficient-Reliable-Local-LLMs

18 octobre 2026

Résumé

Un système de questions-réponses exécuté sur une machine personnelle doit décider, pour chaque question, quel modèle de langage utiliser, combien de documents récupérer, et s'il vaut mieux s'abstenir. Ces trois décisions sont presque toujours étudiées séparément, et leur coût exprimé en prix d'API plutôt qu'en latence ou en énergie mesurées. Nous proposons une politique adaptative simple qui les prend conjointement à partir de deux signaux peu coûteux : la *suffisance des preuves*, calculée à partir du score BM25 du meilleur passage, et la *confiance* du modèle, lue dans les probabilités des tokens générés. Nous l'évaluons sur un Apple M3 Pro avec deux modèles Qwen quantifiés en 4 bits (1,5 et 4 milliards de paramètres), sur un benchmark de 630 questions dérivé de SQuAD 2.0 et HotpotQA, avec des critères de succès fixés avant les expériences et un jeu de test utilisé une seule fois. Sur 450 questions de test, la politique atteint la qualité de la référence « grand modèle + RAG » (score 0,550 contre 0,537 ; non-infériorité démontrée), avec moins de réponses fausses (risque 0,40 contre 0,42), et dépasse deux routeurs de la littérature fondés sur la question seule (0,489 et 0,517). Elle ne réduit toutefois la latence que de 10 % et l'énergie de 9 %, au lieu des 20 % visés : **le critère principal n'est pas atteint**. L'analyse attribue cet échec au réglage d'un seuil d'abstention absolu sur un trop petit jeu de développement. Nous détaillons les points faibles de l'étude et un programme d'améliorations concret.

Mots-clés : LLM locaux, routage de modèles, RAG adaptatif, abstention, efficacité énergétique, évaluation pré-enregistrée.

1 Introduction

Les modèles de langage (LLM) de quelques milliards de paramètres, quantifiés, tournent désormais sur un ordinateur portable. Les exécuter localement protège les données et supprime la dépendance à un service distant, mais chaque requête a un coût direct : temps d'attente, mémoire, énergie. La génération augmentée par récupération (RAG) améliore la factualité (Gao et al., 2023), mais elle allonge le prompt et donc la latence. Un grand modèle répond mieux qu'un petit, mais plus lentement. Et lorsque les documents ne contiennent pas la réponse, le meilleur choix est souvent de ne pas répondre (Wen et al., 2024).

Appliquer la configuration la plus coûteuse à toutes les questions gaspille des ressources sur les questions faciles ; appliquer la moins coûteuse dégrade la qualité sur les questions difficiles. D'où la question de recherche :

> *Comment allouer automatiquement, et de manière mesurable, les ressources de calcul et de recherche documentaire à chaque requête, afin de réduire le coût d'un système LLM local sans dégrader significativement la qualité, la factualité et la fiabilité des réponses ?*

Elle se décline en quatre questions :

QR1 Un signal peu coûteux permet-il de choisir le modèle et le niveau de recherche sans perte de qualité ?

QR2 Peut-on décider de l'abstention de manière fiable, en combinant incertitude du modèle et qualité des preuves ?

QR3 Quel compromis qualité–latence–énergie obtient-on sur matériel local, une fois le coût des signaux compté ?

QR4 Les gains sont-ils homogènes selon le type de question ?

Contributions.

1. Une revue de littérature qui montre que le choix du modèle, la décision de recherche et l'abstention sont étudiés séparément, avec un coût rarement mesuré en local (section 2).
2. Un **benchmark et un protocole pré-enregistrés** sur matériel local : critères de succès écrits avant les expériences, jeu de test utilisé une seule fois, latence mesurée sur trois passes, énergie mesurée par `powermetrics` (sections 3 et 4).
3. Des **méthodes de référence** : six stratégies fixes et deux routeurs inspirés de RouteLLM et d'Adaptive-RAG (section 5).
4. Une **politique adaptative conjointe** (modèle × nombre de passages × abstention), fondée sur deux signaux quasi gratuits et interprétables (section 6).
5. Une **évaluation finale** honnête, qui aboutit à un résultat partiellement négatif, et un **programme d'améliorations** fondé sur l'analyse de ses causes (sections 7 à 9).

2 État de l'art

Méthode de revue. Les sources consultées sont arXiv, ACL Anthology, OpenReview, Google Scholar, Semantic Scholar et les actes de NeurIPS, ICLR, AAAI et MLSys. Les requêtes combinent *LLM routing*, *model cascade*, *adaptive retrieval*, *adaptive RAG*, *uncertainty estimation*, *selective prediction*, *abstention* et *energy-aware LLM inference*. Sont retenus les travaux qui adaptent, requête par requête, le choix du modèle, la recherche ou la décision de répondre, avec une évaluation expérimentale, ainsi que les mesures de coût d'inférence et les jeux de données utiles. Sur 51 références identifiées (recherche initiale, *snowballing* et recherche croisée sur arXiv), 46 ont été retenues et classées par thème.

2.1 Routage et cascades de modèles

Router, c'est choisir avant la génération le modèle qui traitera une requête ; une *cascade* interroge d'abord un modèle peu coûteux et n'escalade vers un plus puissant que si sa réponse semble peu fiable. FrugalGPT (Chen et al., 2023) enchaîne des modèles de coût croissant et s'arrête dès qu'un scoreur appris juge la réponse fiable. AutoMix (Aggarwal et al., 2023) escalade selon une auto-vérification, et Yue et al. (2023) selon la cohérence entre réponses. Hybrid LLM (Ding et al., 2024) prédit la difficulté de la requête pour envoyer les plus simples au petit modèle ; RouteLLM (Ong et al., 2024) apprend des routeurs fort/faible à partir de préférences humaines. D'autres approches utilisent un bandit manchot (Nguyen et al., 2024) ou un modèle de récompense (Lu et al., 2023). RouterBench (Hu et al., 2024) propose un cadre d'évaluation commun, et la revue de Varangot-Reille et al. (2026) constate que le coût y est presque toujours un prix d'API. Plus récemment, Fu et al. (2026) utilisent un signal local avant exécution et un coût réel en tokens, et RACER (Hao et al., 2026) apporte des garanties conformes, c'est-à-dire valides sans hypothèse sur la distribution des données.

2.2 RAG adaptatif

Mallen et al. (2023) montrent que la recherche est surtout utile pour les connaissances rares, et peut nuire pour les connaissances courantes. Adaptive-RAG (Jeong et al., 2024) entraîne un classifieur de complexité qui choisit entre aucune, une ou plusieurs étapes de recherche. FLARE (Jiang et al., 2023) et DRAGIN (Su et al., 2024) déclenchent la recherche pendant la génération lorsque le modèle devient incertain. Self-RAG (Asai et al., 2024) apprend au modèle à juger lui-même la pertinence des passages ; Corrective RAG (Yan et al., 2024) évalue la qualité des documents et les corrige ou les filtre ; Tiny-Critic RAG (Wu et al., 2026) confie ce jugement à un petit modèle. Moskvoretskii et al. (2025) comparent 27 méthodes et montrent que des signaux d'incertitude simples rivalisent avec des pipelines complexes. RAGRouter-Bench (Wang et al., 2026) et Self-Routing RAG (Wu et al., 2025) routent entre paradigmes de RAG, et LeanContext (Arefeen et al., 2024) réduit le contexte récupéré pour en diminuer le coût.

2.3 Incertitude et abstention

Les probabilités internes des grands modèles sont raisonnablement calibrées (Kadavath et al., 2022). L'entropie sémantique (Kuhn et al., 2023) et SelfCheckGPT (Manakul et al., 2023) exploitent la cohérence entre plusieurs réponses échantillonnées, au prix de plusieurs générations ; la confiance exprimée en mots n'est fiable qu'après un apprentissage dédié (Lin et al., 2022). Kamath et al. (2020) formalisent la réponse sélective et ses deux mesures : la *couverture* (part des questions auxquelles le système répond) et le *risque* (part d'erreurs parmi ces réponses). La revue de Wen et al. (2024) montre que l'abstention reste rarement intégrée aux systèmes de RAG.

2.4 Coût et énergie de l'inférence

Samsi et al. (2023) mesurent l'énergie de l'inférence de LLaMA sur GPU de centre de données, Luccioni et al. (2022) l'empreinte carbone de BLOOM, et Luccioni et al. (2024) le coût énergétique de différentes tâches. Les mesures sur matériel personnel restent rares.

2.5 Positionnement

Travail	**Choix du modèle**	**Décision RAG**	**Abstention**	**Signal de décision**	**Coût évalué**	**Local**
FrugalGPT	✓	✗	✗	Scoreur appris (après génération)	Prix API	✗
Hybrid LLM	✓	✗	✗	Difficulté prédite (avant)	Appels au grand modèle	∼
RouteLLM	✓	✗	✗	Préférences humaines	Prix / appels	✗
AutoMix	✓	✗	✗	Auto-vérification (après)	Calcul	✗
RACER	✓	✗	✓	Score calibré	Taille d'ensemble	✗
Adaptive-RAG	✗	✓	✗	Complexité (classifieur)	Temps, étapes	✗
Corrective RAG	✗	✓	✗	Qualité des documents	–	✗
Moskvoretskii et al.	✗	✓	✗	Incertitude	Appels, récupérations	✗
RAGRouter-Bench	∼	✓	✗	Compatibilité requête–corpus	Ressources	✗
Kamath et al.	✗	✗	✓	Calibrateur appris	–	✗
Ce travail	✓	✓	✓	**Preuve BM25 + confiance, coût compté**	**Latence, tokens, énergie**	✓

TABLE 1 – Positionnement des travaux les plus proches. ✓ : traité ; ✗ : non traité ; ∼ : partiellement.

Quatre limites communes ressortent : (i) les trois décisions sont prises *isolément* ; (ii) le coût est surtout *financier*, rarement mesuré en local ; (iii) le *coût des signaux* eux-mêmes (plusieurs générations, un modèle juge) est rarement retranché du gain ; (iv) l'évaluation est parfois optimiste :

Li et al. (2026) montrent que certains gains disparaissent face à un simple routeur « question seule », et une moyenne globale peut masquer des pertes sur certaines catégories. Ce travail vise ces quatre points, avec quatre hypothèses :

H1 Un signal peu coûteux, calculé avant génération, oriente une part significative des requêtes vers le petit modèle sans perte de qualité au-delà de la marge fixée.

H2 Combiner incertitude du modèle et suffisance des preuves améliore le compromis couverture/risque par rapport à chaque signal seul.

H3 La politique conjointe domine les stratégies fixes sur la courbe qualité–coût, coût de ses signaux inclus.

H4 Les gains varient selon la catégorie de question.

3 Benchmark et protocole

3.1 Benchmark

Sources. SQuAD 2.0, jeu de développement (Rajpurkar et al., 2018) : 35 articles de Wikipédia, avec des questions à réponse et des questions sans réponse rédigées pour ressembler aux premières. HotpotQA, jeu de développement en configuration *distractor* (Yang et al., 2018) : questions qui exigent de combiner deux pages Wikipédia. Les deux jeux sont sous licence CC BY-SA 4.0 ; leur empreinte SHA-256 est vérifiée à la construction.

Corpus. Le corpus est la « bibliothèque » dans laquelle la recherche puise. Il réunit les 1 204 paragraphes de SQuAD 2.0 et les 2 089 paragraphes des questions HotpotQA retenues, soit **3 293 passages**. Toutes les questions interrogent ce même corpus.

Catégories et séparation. Trois catégories équilibrées (tableau 2). Les questions SQuAD sont séparées *par article* (30 % des articles pour le développement), afin qu'aucun paragraphe ne serve à la fois au réglage et au test ; les questions HotpotQA sont tirées sans recouvrement (graine 2026). Une quatrième catégorie, les questions ambiguës, était prévue mais n'a pas été annotée.

Catégorie	Source	Définition	Attendu	Dev	Test
Simple	SQuAD 2.0	La réponse se trouve dans un seul passage	Répondre	60	150
Complexe	HotpotQA	La réponse exige de combiner deux passages	Répondre	60	150
Non couverte	SQuAD 2.0	Le passage d'origine ne contient pas la réponse	S'abstenir	60	150
Total				180	450

TABLE 2 – Catégories du benchmark et répartition entre développement et test.

3.2 Environnement et génération

Apple M3 Pro (12 cœurs, 18 Go de mémoire unifiée), macOS, Python 3.12, MLX 0.32 et mlx-lm 0.32. Deux modèles quantifiés en 4 bits, préchargés en mémoire : **Qwen2.5-1.5B-Instruct** (« petit ») et **Qwen3-4B-Instruct-2507** (« grand »). Les révisions exactes des modèles sont consignées. Le décodage est glouton (température 0), limité à 32 tokens : les sorties sont déterministes. Les prompts sont identiques pour tous les systèmes et contiennent une consigne d'abstention explicite : répondre exactement `UNANSWERABLE` si la réponse est inconnue ou absente des passages (annexe A). Trois requêtes de chauffe précèdent chaque série.

3.3 Métriques

— **Score global** par question : F1 lexical entre la réponse et la meilleure réponse de référence si une réponse est attendue ; 1 si le système s'abstient sur une question non couverte, 0 sinon.

C'est la métrique principale.

— **Réponse correcte** : F1 $\geq$ 0,5. **Couverture** : part des questions auxquelles le système répond ; **risque** : part de réponses incorrectes parmi elles (Kamath et al., 2020).

— **Latence** de bout en bout (recherche, signaux et génération), avec une horloge monotone ; moyenne et 95e centile (P95). **Tokens** du prompt et générés.

— **Énergie** : puissance CPU + GPU + ANE relevée par `powermetrics` toutes les 200 ms, intégrée sur la durée de chaque série.

Analyse statistique. Intervalles de confiance à 95 % par bootstrap (10 000 rééchantillonnages des questions). Les différences entre systèmes sont calculées question par question et rééchantillonnées ensemble (bootstrap apparié). Les résultats par catégorie sont systématiquement rapportés.

3.4 Critères de succès pré-enregistrés

Les critères suivants ont été écrits et publiés avant toute expérience.

Critère principal. Sur le jeu de test, par rapport à la stratégie B (grand modèle + RAG) : (i) latence moyenne réduite d'**au moins 20 %** ; (ii) perte de score global d'**au plus 2 points**. La *non-infériorité* n'est revendiquée que si la borne supérieure de l'intervalle de confiance à 95 % de la perte est inférieure à 2 points.

Critères secondaires. Fiabilité : risque inférieur à celui de B. Contrôle : meilleur score que le routeur fondé sur la seule question. Robustesse : aucune catégorie ne perd plus de 5 points.

Résultat négatif. S'il n'est pas atteint, le résultat est rapporté tel quel, avec l'analyse de ses causes.

Discipline d'évaluation. Tout réglage se fait sur le jeu de développement. Le protocole a été figé dans une configuration versionnée (étiquette Git `protocole-v1.0`), puis la politique (étiquette `politique-v1.0`), avant toute lecture du jeu de test. Celui-ci n'a été exécuté qu'une fois.

4 Étude pilote

Un pilote a exécuté quatre stratégies fixes sur le jeu de développement, avec une recherche dense (embeddings MiniLM multilingues, similarité cosinus). Il valide la chaîne complète et oriente la suite.

Stratégie	Score	Couv.	Risque	Latence	Tokens	Énergie / q.
A – grand, sans RAG	0,313	0,31	0,82	0,25 s	62	3,8 J
B – grand + RAG	**0,424**	0,56	0,53	0,99 s	579	22,8 J
C – petit, sans RAG	0,233	0,46	0,93	0,13 s	84	1,9 J
D – petit + RAG	0,298	0,78	0,71	0,39 s	601	8,6 J
Oracle	*0,623*	–	–	*0,30 s*	–	–

TABLE 3 – Pilote sur le jeu de développement (180 questions, recherche dense). Oracle : pour chaque question, la stratégie la moins coûteuse qui obtient le meilleur score.

Enseignements.

1. **Les écarts de coût sont importants** : B est 7,8 fois plus lente que C et coûte 12 fois plus d'énergie. Le RAG augmente à la fois la durée et la puissance appelée (24 W contre 17 W).

2. **Aucune stratégie fixe n'est la meilleure partout** : B l'emporte sur les questions simples, D sur les complexes, A sur les non couvertes. L'oracle (0,62 en 0,30 s) montre une marge réelle pour une politique adaptative.
3. **L'abstention laissée au seul prompt est mal calibrée** : le petit modèle avec RAG répond presque toujours (risque 0,71), le grand sans RAG s'abstient sur 69 % des questions. La décision d'abstention doit donc relever de la politique.
4. **La recherche est le maillon faible** pour les questions complexes : les deux passages justificatifs ne sont récupérés ensemble que dans 22 % des cas avec 3 passages.

5 Méthodes de référence

5.1 Choix de la recherche documentaire

Trois méthodes ont été comparées sur le jeu de développement : la recherche **dense** du pilote, une recherche lexicale **BM25** ($k_1 = 1{,}5$, $b = 0{,}75$) et une **hybride** qui fusionne les deux classements par la somme des inverses des rangs.

	Simple			**Complexe**			
Méthode	@1	@3	@6	@1	@3	@6	**Temps / requête**
Dense	0,48	0,60	0,70	0,00	0,22	0,35	7,4 ms
BM25	**0,65**	**0,82**	**0,87**	0,00	**0,37**	**0,50**	**0,2 ms**
Hybride	0,60	0,72	0,80	0,00	0,33	0,48	7,1 ms

Table 4 – Rappel de la recherche (développement). Pour une question complexe, les deux passages justificatifs doivent être récupérés : le rappel@1 est nul par construction.

BM25 est retenu : il domine sur les questions qui ont une réponse, à chaque profondeur, et il est 40 fois plus rapide. La recherche hybride retrouve plus souvent le paragraphe d'origine des questions non couvertes (rappel@3 de 0,90 contre 0,83), un critère moins important pour elles puisque le comportement attendu est l'abstention. Les questions, riches en noms propres repris du texte source, favorisent la recherche lexicale. Ce choix a été figé avant la suite (`protocole-v1.0`).

5.2 Six stratégies fixes

Stratégie	**Score**	**IC 95 %**	**Simple**	**Complexe**	**Non couv.**	**Couv.**	**Risque**	**Latence**
A – grand, 0 passage	0,312	[0,248 ; 0,378]	0,07	0,09	0,78	0,31	0,82	0,25 s
B – grand, 3 passages	**0,541**	[0,472 ; 0,610]	0,68	0,36	0,58	0,62	0,39	0,92 s
C – petit, 0 passage	0,233	[0,174 ; 0,294]	0,05	0,07	0,58	0,46	0,93	0,13 s
D – petit, 3 passages	0,330	[0,267 ; 0,397]	0,48	0,29	0,22	0,83	0,69	0,37 s
E – grand, 6 passages	0,570	[0,501 ; 0,637]	0,70	0,45	0,57	0,67	0,40	1,63 s
F – petit, 6 passages	0,372	[0,308 ; 0,439]	0,53	0,37	0,22	0,89	0,66	0,62 s

Table 5 – Stratégies fixes avec BM25 (développement, 180 questions). Simple, Complexe et Non couv. : score global par catégorie.

Avec BM25, la référence B passe de 0,424 (pilote) à 0,541 : la barre du critère de succès est *plus exigeante*. E améliore B de 2,9 points, surtout sur les questions complexes, mais coûte 76 % de latence en plus ; l'écart n'est pas significatif. Une recherche plus large n'est donc utile que pour une partie des questions, ce qui justifie de la décider au cas par cas.

5.3 Routeurs de référence

Les deux routeurs ne lisent que le *texte de la question*, représenté par un TF-IDF sur les mots et paires de mots et quelques indicateurs simples (longueur, mots en majuscule, chiffres, mot interrogatif). La génération étant déterministe, la réponse d'un routeur est celle de la stratégie qu'il choisit ; son temps de calcul est ajouté à la latence.

— **R1 – question seule** (contrôle de Li et al., 2026, esprit RouteLLM et Hybrid LLM) : une régression ridge par stratégie prédit son score $\hat{y}_s(q)$, et le routeur choisit $\arg\max_s \hat{y}_s(q) - \lambda\, \bar{t}_s$, où $\bar{t}_s$ est la latence moyenne de s. Le point retenu ($\lambda = 0{,}113$) est le meilleur réglage au moins 20 % plus rapide que B.

— **R2 – Adaptive-RAG simplifié** (Jeong et al., 2024) : une régression logistique prédit la complexité et choisit, avec le grand modèle, entre aucune recherche (A), 3 passages (B) et 6 passages (E). L'étiquette d'apprentissage est la stratégie la plus simple qui répond correctement.

Méthode	Score	Écart à B	IC 95 % de l'écart	Latence	Couv.	Risque
B – référence	0,541	–	–	0,92 s	0,62	0,39
R1 – question seule	0,425	−11,6	[−18,5; −4,8]	0,74 s	0,58	0,57
R2 – Adaptive-RAG simplifié	0,522	−1,9	[−7,4; +3,3]	0,85 s	0,49	0,39
Oracle (A–F)	*0,762*	+22,1	[+16,5; +27,8]	*0,40 s*	0,58	0,19

TABLE 6 – Routeurs de référence, validation croisée en 5 plis sur le développement.

Lecture. Contraint d'être 20 % plus rapide que B, R1 perd 11,6 points : le texte de la question ne permet pas de prédire de façon fiable quelle stratégie réussira. R2 conserve la qualité de B mais ne gagne que 8 % de latence, et ne choisit jamais le petit modèle. L'oracle montre qu'on pourrait à la fois gagner 22 points et diviser la latence par plus de deux. **Conclusion de cette étape : une méthode adaptative doit utiliser d'autres signaux que la question**, comme ce que la recherche a trouvé ou l'incertitude du modèle sur sa réponse.

6 Politique adaptative

6.1 Signaux

Suffisance des preuves. La recherche BM25 renvoie le score S_1 du meilleur passage. On le normalise par le score maximal atteignable pour la question q :

$$\text{preuve}(q) \;=\; \frac{S_1}{\sum_{t \in q} \text{idf}(t)\,(k_1 + 1)} \;\in\; [0, 1].$$

Une valeur proche de 1 indique qu'un passage contient tous les mots importants de la question. Ce signal est un sous-produit de la recherche (0,2 ms) : il ne coûte rien.

Confiance du modèle. Pendant le décodage glouton, on relève la probabilité de chaque token produit, et on en retient la moyenne géométrique ou le minimum. Le décodage n'est pas modifié : les réponses sont identiques à celles des stratégies fixes, ce qui a été vérifié.

Signal	Ce qu'on cherche à prédire	AUC
Confiance du petit modèle, 3 passages	Sa réponse est correcte	0,74
Confiance du grand modèle, 3 passages	Sa réponse est correcte	0,63
Confiance du grand modèle, 6 passages	Sa réponse est correcte	0,70
Preuve BM25	La réponse de B est correcte	0,55
Preuve BM25 (faible)	La question est non couverte	0,62

TABLE 7 – Pouvoir prédictif des signaux (développement). AUC : 0,5 = hasard, 1 = parfait.

Les signaux sont informatifs, mais modérément : aucun ne dépasse une AUC de 0,74.

6.2 Politique

Pour chaque question :

1. **Recherche BM25** de 6 passages et calcul de la preuve.
2. **Abstention précoce** : si preuve(q) $< \tau_0$, réponse `UNANSWERABLE` immédiate, sans aucune génération.
3. **Cascade** (optionnelle) : le petit modèle répond avec 3 passages ; si sa confiance dépasse τ_p, sa réponse est retenue.
4. **Grand modèle** sinon, avec 6 passages si preuve(q) $< \tau_k$, 3 sinon.
5. **Abstention finale** si le grand modèle s'abstient ou si sa confiance est inférieure à τ_a.

Le coût de chaque étape exécutée, signaux compris, est compté dans la latence.

6.3 Réglage

Simulation exacte. Les sorties étant déterministes, la sortie de la politique pour une question est celle de l'étape où elle s'arrête. Les sorties de chaque étape ont été calculées une fois ; la politique est alors simulée exactement, latences comprises, pour tout réglage.

Objectif. Parmi 246 510 réglages (type de confiance, seuils τ_p, τ_k, τ_0, τ_a, cascade ou non), on retient celui qui maximise le score global sous la contrainte d'une latence moyenne au plus égale à 78 % de celle de B. Les 2 points de marge par rapport au critère de 80 % sont censés protéger contre la variabilité entre jeux de questions. Le score du réglage retenu étant optimiste, on l'estime aussi par validation croisée en 5 plis.

Démarche de conception. Deux conceptions ont été essayées, sur le développement uniquement. La première, une *cascade seule* (petit modèle, puis grand si incertain, puis abstention), n'atteint que 0,395 sous la contrainte de latence (−14,6 points) : le petit modèle se trompe trop souvent, et chaque escalade paie deux générations. La seconde ajoute l'*abstention précoce* sur preuve faible ; le réglage, qui pouvait toujours activer la cascade, l'a écartée.

Politique figée. $\tau_0 = 0{,}265$; $\tau_k = 0{,}286$; abstention finale si la probabilité minimale d'un token est inférieure à $\tau_a = 0{,}40$; pas de cascade. Dit simplement : *si les documents ne contiennent manifestement pas les mots de la question, s'abstenir sans rien générer ; sinon, interroger le grand modèle, avec plus de passages si la preuve est limite, et s'abstenir s'il hésite.*

6.4 Résultats sur le développement

Méthode	Score	Écart à B	Latence (gain)	Couv.	Risque
B – référence	0,541	–	0,92 s	0,62	0,39
R1 – question seule	0,425	−11,6	0,74 s (20 %)	0,58	0,57
R2 – Adaptive-RAG simplifié	0,522	−1,9	0,85 s (8 %)	0,49	0,39
Politique (réglage figé)	**0,552**	**+1,1**	**0,71 s (23 %)**	0,48	**0,34**
Politique (validation croisée)	0,520	−2,1	0,72 s (22 %)	0,44	0,35

TABLE 8 – Politique adaptative sur le développement (180 questions).

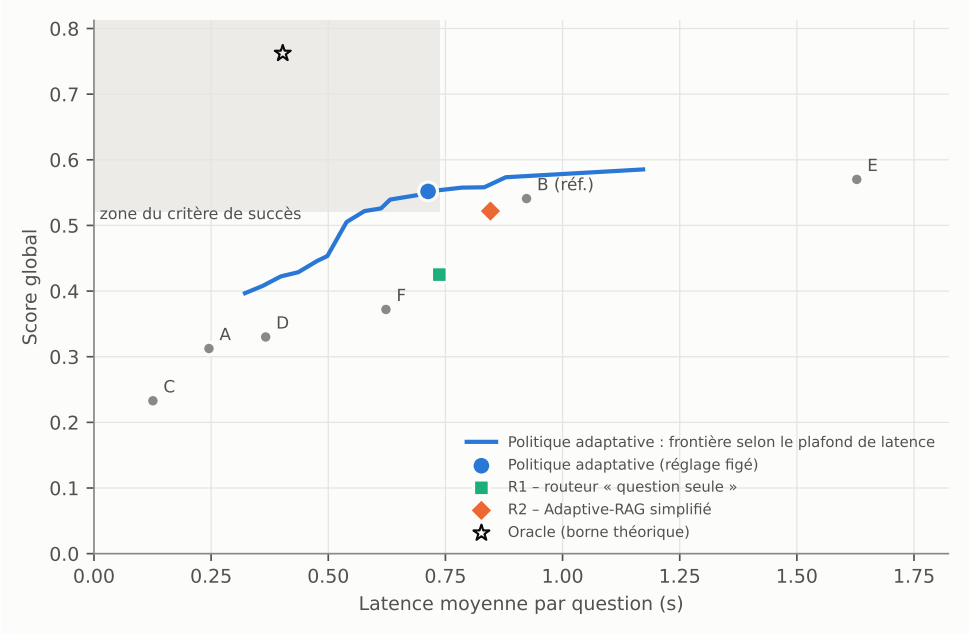


FIGURE 1 – Compromis qualité–latence sur le développement. La courbe bleue donne, pour chaque plafond de latence, le meilleur score atteignable par la politique ; le point bleu est le réglage figé. La zone grisée correspond au critère de succès.

Sur le développement, la politique remplit le critère principal (+1,1 point, −23 % de latence) et dépasse R1 et R2. Deux signaux d'alerte étaient toutefois visibles : la validation croisée estimait une perte de 2,1 points, à la limite du critère, et les questions simples perdaient 14 points (le critère de robustesse n'était donc déjà pas satisfait), en échange d'un gain de 21,7 points sur les questions non couvertes.

7 Évaluation finale sur le jeu de test

7.1 Exécution

Les six stratégies fixes et la politique ont été exécutées sur les 450 questions du test, en **trois passes complètes**, avec un ordre des méthodes tiré au hasard dans chaque passe pour répartir l'effet de l'échauffement de la machine. La latence retenue est la médiane des trois passes pour chaque question. La politique est exécutée réellement de bout en bout. R1 et R2, entraînés sur tout le développement, sont appliqués aux sorties du test. L'énergie a été mesurée pendant toute l'évaluation (30 630 relevés ; puissance au repos 2,7 W).

Deux contrôles valident l'exécution : les réponses sont **identiques dans les trois passes** pour toutes les méthodes, et la politique exécutée **reproduit exactement sa simulation** (accord de

100 % sur le score, latence à 0,001 s près).

7.2 Résultats principaux

Méthode	Score	IC 95 %	Écart à B	Latence	P95	Couv.	Risque
A – grand, sans RAG	0,310	[0,269 ; 0,351]	−22,7	0,27 s	0,52 s	0,45	0,76
B – grand, 3 passages (réf.)	**0,537**	[0,494 ; 0,581]	–	**0,87 s**	1,26 s	0,73	0,42
C – petit, sans RAG	0,223	[0,187 ; 0,261]	−31,4	0,13 s	0,25 s	0,51	0,93
D – petit, 3 passages	0,414	[0,372 ; 0,457]	−12,3	0,35 s	0,53 s	0,88	0,57
E – grand, 6 passages	0,567	[0,523 ; 0,610]	+3,0	1,57 s	2,23 s	0,75	0,40
F – petit, 6 passages	0,398	[0,357 ; 0,440]	−13,9	0,61 s	0,87 s	0,94	0,60
R1 – question seule	0,489	[0,445 ; 0,534]	−4,7	0,74 s	1,61 s	0,68	0,48
R2 – Adaptive-RAG simplifié	0,517	[0,473 ; 0,561]	−2,0	0,82 s	1,39 s	0,68	0,44
Politique adaptative	**0,550**	[0,505 ; 0,594]	**+1,3**	**0,79 s**	1,38 s	0,64	**0,40**
Oracle	*0,797*	[0,762 ; 0,831]	+26,0	*0,36 s*	1,09 s	0,64	0,18

TABLE 9 – Résultats sur le jeu de test (450 questions). Latence : moyenne par question de la médiane des trois passes.

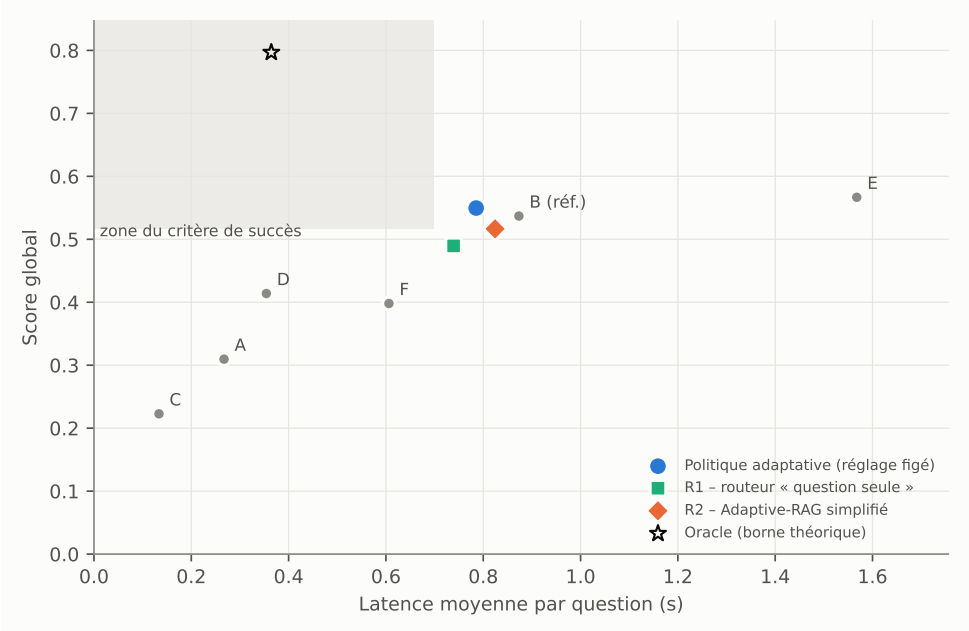


FIGURE 2 – Compromis qualité–latence sur le test. La politique (en bleu) est au-dessus de B, mais à droite de la zone du critère de succès.

7.3 Verdict sur les critères pré-enregistrés

Critère	Mesure	Atteint
Latence moyenne réduite d'au moins 20 %	−10,1 % (0,786 contre 0,873 s)	✗
Perte de score ≤ 2 points	−1,3 (gain)	✓
Non-infériorité (borne haute de l'IC de la perte < 2)	[−4,1 ; +1,6]	✓
Critère principal		✗
Risque inférieur à B	0,395 contre 0,421	✓
Meilleur score que R1 et R2	0,550 contre 0,489 et 0,517	✓
Aucune catégorie ne perd plus de 5 points	simple −7,6 ; complexe −3,3 ; non couverte +14,7	✗

TABLE 10 – Verdict sur le jeu de test.

L'hypothèse principale est **rejetée dans sa forme pré-enregistrée** : la politique conserve la qualité de B avec moins d'erreurs, mais n'économise que la moitié de la latence visée.

7.4 Par catégorie et choix de la politique

Méthode	Simple	Complexe	Non couverte
B – référence	0,737	0,468	0,407
E – grand, 6 passages	0,730	0,537	0,433
R1	0,562	0,453	0,453
R2	0,635	0,455	0,460
Politique adaptative	0,661 (−7,6)	0,435 (−3,3)	**0,553** (+14,7)
Oracle	0,842	0,682	0,867

TABLE 11 – Score par catégorie sur le test (150 questions chacune). Entre parenthèses : écart à B.

Sur 450 questions, la politique a déclenché 66 abstentions précoces (15 %) et a fait répondre le grand modèle 359 fois avec 3 passages et 25 fois avec 6 ; 98 de ces réponses ont ensuite été remplacées par une abstention finale. L'abstention précoce touche surtout les questions non couvertes (36 sur 150), mais aussi 18 questions simples et 12 complexes, d'où les pertes sur ces catégories.

7.5 Ablations

	Test		**Développement**	
Variante (chacune réglée sur le développement)	Score	Latence	Score	Latence
Politique complète	**0,550**	0,79 s	0,552	0,71 s
Sans recherche adaptative (toujours 3 passages)	0,547	0,74 s	0,548	0,68 s
Sans abstention finale	0,547	0,79 s	0,546	0,71 s
Sans abstention précoce (cascade petit → grand)	0,464	0,64 s	0,395	0,70 s
Sans aucune abstention	0,457	0,64 s	0,389	0,70 s

TABLE 12 – Ablations.

L'abstention précoce fondée sur la preuve est la composante décisive (−8,6 points sans elle sur le test). La recherche adaptative et l'abstention finale n'apportent chacune que 0,3 point, dans le bruit statistique ; toujours utiliser 3 passages serait même plus rapide (−15 %) pour un score équivalent.

7.6 Énergie

Méthode	Énergie / question	Écart à B	Puissance moyenne
C – petit, sans RAG	1,8 J	−90 %	16,4 W
A – grand, sans RAG	3,9 J	−80 %	17,0 W
D – petit, 3 passages	7,3 J	−62 %	23,2 W
F – petit, 6 passages	13,5 J	−30 %	24,9 W
Politique adaptative	**17,5 J**	**−9 %**	24,8 W
B – grand, 3 passages (réf.)	19,3 J	–	24,7 W
E – grand, 6 passages	37,7 J	+95 %	26,4 W

TABLE 13 – Énergie de la puce (CPU + GPU + ANE) par question sur le test, moyenne des trois passes.

La puissance appelée varie peu (16 à 26 W) : l'énergie suit la durée de calcul, donc le nombre de tokens traités. L'économie d'énergie de la politique (−9 %) suit sa réduction de latence (−10 %).

7.7 Du développement au test

	Développement (180)		Test (450)	
Méthode	Score	Latence	Score	Latence
B – référence	0,541	0,92 s	0,537	0,87 s
Politique adaptative	0,552	0,71 s (−23 %)	0,550	0,79 s (−10 %)
R1	0,425	0,74 s	0,489	0,74 s
R2	0,522	0,85 s	0,517	0,82 s
Oracle	0,762	0,40 s	0,797	0,36 s
Abstentions précoces	45 / 180 (25 %)		66 / 450 (15 %)	

TABLE 14 – Comparaison entre développement et test.

La qualité s'est généralisée, mieux même que ne le prévoyait la validation croisée (0,550 au lieu de 0,520). **Le gain de latence ne s'est pas généralisé** : le seuil d'abstention précoce, réglé sur 60 questions par catégorie, coupait 25 % des questions de développement mais n'en coupe que 15 % sur le test, un écart d'environ trois erreurs-types, attribuable à la variabilité d'échantillonnage d'un petit jeu de réglage. Moins d'abstentions précoces signifie plus d'appels au grand modèle, donc une latence plus proche de celle de B.

8 Discussion

Retour sur les hypothèses.

H1 – rejetée. Aucun signal calculé avant génération n'a permis d'orienter des requêtes vers le petit modèle sans perte : R1 perd 4,7 à 11,6 points, et la politique réglée n'utilise jamais le petit modèle.

H2 – non confirmée. La politique a un risque plus faible que B, mais l'apport de la confiance du modèle en plus de la preuve est négligeable (0,3 point, risque inchangé). C'est la preuve seule qui porte le gain.

H3 – partiellement confirmée. La politique n'est dominée par aucune stratégie fixe ni aucun routeur (aucun n'est à la fois meilleur et plus rapide), et elle domine B. Mais son gain de coût est modeste.

H4 – confirmée. Les effets sont très hétérogènes : −7,6 points sur les questions simples, +14,7 sur les questions non couvertes. Une moyenne seule aurait masqué ce compromis.

Enseignements.

1. **La preuve est un meilleur signal de routage que la question.** À latence comparable, la politique dépasse R1 de 6 points et R2 de 3 points. Regarder ce que la recherche a trouvé renseigne mieux que deviner la difficulté d'après la formulation, ce qui rejoint Li et al. (2026).
2. **L'abstention rend l'adaptation rentable.** Ne pas générer sur une preuve faible économise du calcul et évite des erreurs en même temps.
3. **Le petit modèle n'a pas trouvé sa place.** Sa confiance ne signale ses erreurs que modérément (AUC 0,74) et chaque escalade paie deux générations.
4. **La marge reste considérable.** L'oracle atteint 0,797 en 0,36 s : les bons choix existent, c'est leur prédiction qui est difficile.

9 Points faibles et améliorations

Cette section recense les faiblesses de l'étude, de la plus importante à la moins importante, et pour chacune l'amélioration proposée. Le tableau 15 en fait la synthèse.

9.1 Faiblesses de la méthode

F1 – Seuils absolus réglés sur un petit échantillon. C'est la cause directe de l'échec du critère principal. Un seuil sur une valeur absolue de la preuve, réglé sur 60 questions par catégorie, ne déclenche pas la même proportion d'abstentions sur une autre distribution de questions. La validation croisée, faite sur le même petit échantillon, ne pouvait pas le révéler. *Améliorations :* (i) exprimer les seuils comme des *quantiles* ou un *budget* (« s'abstenir sur les x % de questions aux preuves les plus faibles »), ce qui garantit le taux d'abstention, donc la latence ; (ii) calibrer les seuils avec une méthode conforme, qui donne des garanties de risque sans hypothèse sur la distribution (Hao et al., 2026) ; (iii) agrandir le jeu de réglage à au moins 500 questions ; (iv) régler avec une marge de sécurité dérivée de la variabilité observée par bootstrap, et non fixe.

F2 – Perte sur les questions simples (−7,6 points). L'abstention précoce touche aussi 18 questions simples et 12 complexes sur 150 : un score BM25 faible ne signifie pas toujours que la réponse est absente (reformulation, synonymes). La couverture baisse de 0,73 à 0,64. *Améliorations :* (i) remplacer la preuve lexicale par un juge de suffisance léger (re-classeur *cross-encoder* ou petit modèle d'inférence, dans l'esprit de CRAG (Yan et al., 2024) et Tiny-Critic RAG (Wu et al., 2026)) ; (ii) combiner preuve BM25 et preuve dense ; (iii) ajouter au réglage une contrainte par catégorie, de sorte qu'aucune ne perde plus de 5 points.

F3 – Signaux de confiance faibles. Aucun signal ne dépasse une AUC de 0,74, et l'abstention finale n'apporte que 0,3 point. La probabilité des tokens d'une réponse courte informe mal sur sa justesse. *Améliorations :* entropie sémantique (Kuhn et al., 2023) ou cohérence entre quelques échantillons, réservées aux cas limites pour en contenir le coût ; sonde entraînée sur les états internes du modèle (Kadavath et al., 2022).

F4 – Recherche adaptative quasi inutile. Choisir 6 passages au lieu de 3 ne concerne que 6 % des questions et n'apporte rien de mesurable. Or les questions complexes ont un rappel@3 de seulement 0,37 : le problème n'est pas le *nombre* de passages mais la recherche elle-même. *Améliorations :* décomposition ou reformulation des questions multi-étapes, recherche itérative déclenchée par l'incertitude (Jiang et al., 2023; Su et al., 2024), compression du contexte pour réduire le coût des passages supplémentaires (Arefeen et al., 2024).

F5 – Le petit modèle est inutilisé. La politique finale n'utilise qu'un seul modèle : la dimension « choix du modèle » de la question de recherche n'est pas réellement exploitée. *Améliorations :* essayer un petit modèle plus performant ou spécialisé en extraction, ou l'employer pour un rôle moins exigeant (vérification, reformulation de requête) ; décodage spéculatif, où le petit modèle accélère le grand sans changer ses réponses.

F6 – Latence de queue dégradée. La latence P95 de la politique (1,38 s) dépasse celle de B (1,26 s) à cause des réponses à 6 passages. *Amélioration :* inclure le P95 dans l'objectif de réglage, ou plafonner le contexte.

F7 – Écart à l'oracle de 25 points. *Amélioration :* un routeur appris combinant caractéristiques de la question, preuve et confiance, entraîné sur un jeu bien plus grand que 180 questions.

9.2 Faiblesses du protocole

F8 – Benchmark construit et restreint. Questions en anglais tirées de SQuAD 2.0 et HotpotQA, corpus fermé de 3 293 passages ; les questions non couvertes sont des questions adverses de SQuAD 2.0, rédigées pour ressembler aux passages, ce qui n'est pas représentatif de l'usage réel ; la catégorie des questions ambiguës, prévue au protocole, n'a pas été annotée. *Améliorations :* ajouter MuSiQue (Trivedi et al., 2022) et CRAG (Yang et al., 2024), un corpus réel en français, et annoter au moins 50 questions ambiguës.

F9 – Métrique lexicale. Le F1 pénalise les réponses correctes formulées autrement (paraphrase, synonyme, date écrite autrement). *Amélioration :* juge sémantique (modèle d'inférence ou LLM juge), validé sur un échantillon annoté à la main.

F10 – Puissance statistique limitée. Avec 450 questions, l'intervalle de confiance du score est large d'environ ±4,4 points : un écart de 2 points, le seuil du critère, n'est pas détectable avec certitude. *Amélioration :* jeu de test d'au moins 1 500 questions.

F11 – Matériel et mesure uniques. Un seul appareil (M3 Pro, MLX, 4 bits) ; l'énergie est celle de la puce, pas la consommation murale. *Améliorations :* répliquer sur GPU NVIDIA et avec `llama.cpp`, mesurer au wattmètre, rapporter la mémoire.

F12 – Conception explorée sur le développement. Deux conceptions ont été essayées avant de figer la politique ; c'est documenté, mais cela augmente le risque de sur-ajustement au développement. *Amélioration :* pré-enregistrer aussi la *famille* de politiques, et pas seulement les critères. Toute amélioration devra être évaluée sur un **nouveau** jeu de test : celui-ci est désormais consommé.

F13 – Revue de littérature à approfondir. L'état de l'art a d'abord été établi à partir des résumés des articles. *Amélioration :* lecture complète des travaux les plus proches (Adaptive-RAG, Moskvoretskii et al., 2025, RAGRouter-Bench, RACER, Li et al., 2026) pour confirmer le positionnement du tableau 1.

	Faiblesse	Amélioration principale	Priorité
F1	Seuils absolus, petit jeu de réglage	Seuils en quantiles / budget, calibration conforme, ≥ 500 questions	Haute
F2	Perte sur les questions simples	Juge de suffisance léger, contrainte par catégorie	Haute
F10	Puissance statistique	Test $\geq 1\,500$ questions	Haute
F3	Signaux de confiance faibles	Cohérence entre échantillons sur les cas limites	Moyenne
F4	Recherche adaptative inutile	Décomposition des questions multi-étapes	Moyenne
F8	Benchmark restreint	MuSiQue, CRAG, corpus français, questions ambiguës	Moyenne
F9	Métrique lexicale	Juge sémantique validé	Moyenne
F5	Petit modèle inutilisé	Autre petit modèle, décodage spéculatif	Moyenne
F13	Revue fondée sur les résumés	Lecture complète des travaux proches	Moyenne
F6	Latence P95	P95 dans l'objectif de réglage	Basse
F7	Écart à l'oracle	Routeur appris multi-signaux	Basse
F11	Matériel unique	Réplication GPU, wattmètre	Basse
F12	Exploration sur le développement	Pré-enregistrer la famille de politiques	Basse

TABLE 15 – Synthèse des points faibles et des améliorations.

10 Conclusion

Nous avons construit un benchmark et un protocole pré-enregistrés pour évaluer, sur une machine personnelle, l'allocation adaptative du calcul et de la recherche documentaire. Après avoir montré que les routeurs fondés sur la question seule ne suffisent pas, nous avons proposé une politique qui décide conjointement du modèle, du nombre de passages et de l'abstention à partir de la preuve BM25 et de la confiance du modèle. Sur le jeu de test, elle préserve la qualité de la configuration de référence, fait moins d'erreurs et dépasse les deux routeurs de référence ; mais elle ne réduit la latence que de 10 % et l'énergie de 9 %, au lieu des 20 % visés. L'hypothèse principale est donc rejetée dans sa forme pré-enregistrée.

Deux enseignements dépassent ce résultat : sur machine locale, *la qualité des preuves récupérées est un signal de routage plus utile que la question elle-même*, et *savoir s'abstenir est ce qui permet d'économiser sans dégrader la fiabilité*. La prochaine étape est de rendre ce levier stable : exprimer l'abstention comme un budget calibré plutôt qu'un seuil absolu, mieux juger la suffisance des preuves, et valider le tout sur un jeu de test plus grand et neuf.

Reproductibilité. Code, benchmark (avec empreintes des données), configurations figées (étiquettes `protocole-v1.0` et `politique-v1.0`), sorties brutes des trois passes et mesures d'énergie sont publiés dans le dépôt indiqué en première page.

Références

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Manaal Faruqui, and Mausam. AutoMix : Automatically Mixing Language Models. arXiv preprint arXiv :2310.12963, 2023. URL https://arxiv.org/abs/2310.12963.

Md Adnan Arefeen, Biplob Debnath, and Srimat Chakradhar. LeanContext : Cost-efficient domain-specific question answering using LLMs. *Natural Language Processing Journal*, 2024. doi : 10.1016/j.nlp.2024.100065.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG : Learning

to Retrieve, Generate, and Critique through Self-Reflection. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.11511.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT : How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv preprint arXiv :2305.05176, 2023. URL https://arxiv.org/abs/2305.05176.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM : Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2404.14618.

Heming Fu, Shan Lin, Qianqian Xie, and Guojun Xiong. Not All Tokens Are Equal : Inflation-Aware Routing for Agentic LLM Systems. arXiv preprint arXiv :2608.13571, 2026. URL https://arxiv.org/abs/2608.13571.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-Augmented Generation for Large Language Models : A Survey. arXiv preprint arXiv :2312.10997, 2023. URL https://arxiv.org/abs/2312.10997.

Sai Hao, Hao Zeng, Hongxin Wei, and Bingyi Jing. RACER : Risk-Aware Calibrated Efficient Routing for Large Language Models. arXiv preprint arXiv :2603.06616, 2026. URL https://arxiv.org/abs/2603.06616.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench : A Benchmark for Multi-LLM Routing System. arXiv preprint arXiv :2403.12031, 2024. URL https://arxiv.org/abs/2403.12031.

Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG : Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of NAACL-HLT 2024*, 2024. URL https://arxiv.org/abs/2403.14403.

Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active Retrieval Augmented Generation. In *Proceedings of EMNLP 2023*, 2023. URL https://arxiv.org/abs/2305.06983.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language Models (Mostly) Know What They Know. arXiv preprint arXiv :2207.05221, 2022. URL https://arxiv.org/abs/2207.05221.

Amita Kamath, Robin Jia, and Percy Liang. Selective Question Answering under Domain Shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL https://arxiv.org/abs/2006.09462.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic Uncertainty : Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. In *International Conference on Learning Representations (ICLR)*, 2023. URL https://arxiv.org/abs/2302.09664.

Qiaomu Li, Qiuyuan Zhang, and Nong Ming. Beyond the Query : Do Retrieval Signals Improve Adaptive Multimodal RAG Routing ? arXiv preprint arXiv :2609.12437, 2026. URL https://arxiv.org/abs/2609.12437.

Stephanie Lin, Jacob Hilton, and Owain Evans. Teaching Models to Express Their Uncertainty in Words. *Transactions on Machine Learning Research*, 2022. URL https://arxiv.org/abs/2205.14334.

Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. Routing to the Expert : Efficient Reward-guided Ensemble of Large Language Models. arXiv preprint arXiv :2311.08692, 2023. URL https://arxiv.org/abs/2311.08692.

Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. Estimating the Carbon Footprint of BLOOM, a 176B Parameter Language Model. arXiv preprint arXiv :2211.02001, 2022. URL https://arxiv.org/abs/2211.02001.

Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power Hungry Processing : Watts Driving the Cost of AI Deployment ? In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024. URL https://arxiv.org/abs/2311.16863.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When Not to Trust Language Models : Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL https://arxiv.org/abs/2212.10511.

Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT : Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. In *Proceedings of EMNLP 2023*, 2023. URL https://arxiv.org/abs/2303.08896.

Viktor Moskvoretskii, Maria Lysyuk, Mikhail Salnikov, Nikolay Ivanov, Sergey Pletenev, Daria Galimzianova, Nikita Krayko, Vasily Konovalov, Irina Nikishina, and Alexander Panchenko. Adaptive Retrieval Without Self-Knowledge ? Bringing Uncertainty Back Home. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. URL https://arxiv.org/abs/2501.12835.

Quang H. Nguyen, Thinh Dao, Duy C. Hoang, Juliette Decugis, Saurav Manchanda, Nitesh V. Chawla, and Khoa D. Doan. MetaLLM : A High-performant and Cost-efficient Dynamic Framework for Wrapping LLMs. arXiv preprint arXiv :2407.10834, 2024. URL https://arxiv.org/abs/2407.10834.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM : Learning to Route LLMs with Preference Data. arXiv preprint arXiv :2406.18665, 2024. URL https://arxiv.org/abs/2406.18665.

Pranav Rajpurkar, Robin Jia, and Percy Liang. Know What You Don't Know : Unanswerable Questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. URL https://arxiv.org/abs/1806.03822.

Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. From Words to Watts : Benchmarking the Energy Costs of Large Language Model Inference. arXiv preprint arXiv :2310.03003, 2023. URL https://arxiv.org/abs/2310.03003.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. DRAGIN : Dynamic Retrieval Augmented Generation based on the Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2403.10081.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue : Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 2022. URL https://arxiv.org/abs/2108.00573.

Clovis Varangot-Reille, Christophe Bouvard, Antoine Gourru, Mathieu Ciancone, Marion Schaeffer, and François Jacquenet. Doing More with Less : A Survey on Routing Strategies for Resource Optimisation in Large Language Model-Based Systems. *Journal of Artificial Intelligence Research*, 2026. URL https://arxiv.org/abs/2502.00409.

Ziqi Wang, Xi Zhu, Shuhang Lin, Haochen Xue, Minghao Guo, and Yongfeng Zhang. RAGRouter-Bench : A Dataset and Benchmark for Adaptive RAG Routing. arXiv preprint arXiv :2602.00296, 2026. URL https://arxiv.org/abs/2602.00296.

Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. Know Your Limits : A Survey of Abstention in Large Language Models. arXiv preprint arXiv :2407.18418, 2024. URL https://arxiv.org/abs/2407.18418.

Di Wu, Jia-Chen Gu, Kai-Wei Chang, and Nanyun Peng. Self-Routing RAG : Binding Selective Retrieval with Knowledge Verbalization. arXiv preprint arXiv :2504.01018, 2025. URL https://arxiv.org/abs/2504.01018.

Yichao Wu, Penghao Liang, Yafei Xiang, Mengwei Yuan, Jianan Liu, Jing Yang, Xianyou Li, and Weiran Yan. Tiny-Critic RAG : Empowering Agentic Fallback with Parameter-Efficient Small Language Models. arXiv preprint arXiv :2603.00846, 2026. URL https://arxiv.org/abs/2603.00846.

Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective Retrieval Augmented Generation. arXiv preprint arXiv :2401.15884, 2024. URL https://arxiv.org/abs/2401.15884.

Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, Lingkun Kong, Brian Moran, Jiaqi Wang, Yifan Ethan Xu, An Yan, Chenyu Yang, Eting Yuan, Hanwen Zha, Nan Tang, Lei Chen, Nicolas Scheffer, Yue Liu, Nirav Shah, Rakesh Wanga, Anuj Kumar, Wen tau Yih, and Xin Luna Dong. CRAG – Comprehensive RAG Benchmark. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.04744.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA : A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of EMNLP 2018*, 2018. URL https://arxiv.org/abs/1809.09600.

Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. Large Language Model Cascades with Mixture of Thoughts Representations for Cost-efficient Reasoning. arXiv preprint arXiv :2310.03094, 2023. URL https://arxiv.org/abs/2310.03094.

A Prompts

Sans recherche documentaire :

```
Answer the question with a short phrase only. If you do not know the answer
or it cannot be determined, reply exactly: UNANSWERABLE.

Question: {question}
Answer:
```

Avec recherche documentaire :

```
Answer the question using only the passages below, with a short phrase only.
If the passages do not contain the answer, reply exactly: UNANSWERABLE.

[1] {titre}: {texte}
[2] ...
```

```
Question: {question}
Answer:
```